STAT 486 Project: Fall 2026

Teams: Two or three students
Presentations: Tuesday, December 8, or Thursday, December 10, during your assigned class meeting
Final repository due: Thursday, December 10, 2026, at 11:59 PM Mountain Time

Introduction

Use machine learning to investigate a question your team cares about. The project is your chance to spend sustained time with a topic, implement machine learning methods, and learn from and communicate the results. Choose one of three routes: **develop and test something new in an ML method or system**, **participate in a Kaggle competition through a carefully designed pipeline**, or **build a new dataset and use it in a reproducible ML pipeline**. Each route should lead to a claim your team can examine with evidence. A well-investigated negative result can be excellent work.

Choose a route

A. Develop something new

Design, implement, and test a meaningful change to how an ML method or system represents data, learns, evaluates, or makes decisions. "New" means a **team-designed technical idea with a clear purpose**; it does not need to be a publishable algorithm. Start from a simple working reference method, then make one focused change whose effect you can investigate.

If you are new to ML research, these are manageable starting points:

- Design a feature representation or transformation for a particular kind of data, then compare it with ordinary features. For example, would summaries of a time series help more than using its raw measurements?
- Create a rule for handling missing values, noisy labels, class imbalance, or unequal error costs, and test whether it improves the outcome you care about.
- Build a small component such as a custom distance, sampler, cost-aware decision rule, calibration method, or way to combine predictions. Explain why the component fits your problem.
- Reproduce an accessible published method, then add your own component or adapt it to a new constraint and test the change.

These ideas can be modest in code size while still producing a deep investigation. Make the new component explainable and test it on a small or synthetic case before relying on the final result. Compare it with the reference under the same evaluation conditions, and look for a setting where it fails or matters less. Discuss an uncertain idea with the instructor early.

B. Enter a Kaggle competition

Choose a competition the instructor approves, whose rules allow your team and planned data use, and whose data and compute needs are feasible by December 10. Build **at least two meaningfully different pipelines and make at least two valid Kaggle submissions, one generated by each pipeline**. Each pipeline must have a documented path from competition data through preparation, local validation, training, and inference to its submission file. The pipelines may share code, but each needs its own configuration and reproducible run command. Changing only a random seed or a minor hyperparameter setting does not make a second pipeline. A new algorithm is not required; a substantive change might involve feature representation, missing-data treatment, model training, calibration, or inference.

Begin with a simple baseline pipeline and a local validation score you trust. Use what you learn to design the second pipeline. In your README, report, and presentation, explain **what changed and why**, compare the local and Kaggle results for both submissions, and discuss any improvement or degradation. Use the same local evaluation where possible; if you change it, explain which scores remain comparable. Investigate a useful error pattern or discrepancy between local results and the public leaderboard. Rank is context, not the measure of project quality, and a weaker second result can still teach you something valuable. Follow competition rules for data and code sharing, and credit public notebooks or solutions you use.

C. Build a dataset and an ML pipeline

Create a useful new dataset from one or more sources, then use it to investigate an ML question. You might combine open datasets, collect records through an API, or scrape sites that permit your planned collection and use. The dataset can be your main technical contribution; you may use standard ML methods to analyze it. Downloading a file and fitting a model is a starting point, but this route calls for **team-designed collection, integration, cleaning, or labeling work** that makes the resulting data more useful or reliable.

Decide what one observation represents and what the dataset should contain. Build a repeatable path from permitted source data to the cleaned dataset. **Commit the scripts you use to scrape or otherwise collect the reported data, along with the scripts that merge and clean it, to your GitHub repository.** Check important assumptions such as join coverage, duplicates, missing values,

label consistency, or changes in a source over time. Record source locations, access or license terms, collection dates or versions, and the transformations you made. If a live source can change, preserve a permitted snapshot or sample and document how the reported dataset was produced. Do not publish raw data that you lack permission to redistribute.

Apply ML to a question the new data can help answer, using a suitable reference method and evaluation plan. Show both that the dataset is fit for your purpose and what the ML results reveal. Useful evidence might compare results before and after adding a source or cleaning rule, or show how missingness or coverage limits a conclusion; an accuracy gain is not required. For example, a team might join permitted transit and weather records by place and time, measure how many records match, then test whether the added weather information helps predict delays. A custom estimator is optional; the depth here comes from the dataset design, quality checks, and reproducible data-to-result pipeline. One source is enough if your collection and preparation add substantial value.

All three routes have the same schedule, final deliverables, and overall grading standards. In your project plan, name the route and the specific contribution or pipeline decision you intend to investigate. A justified pivot is welcome when your evidence points elsewhere.

What a project needs to show

Your final work should tell a coherent story from **question to evidence to conclusion**. The form of the investigation is flexible, but a reader should be able to find these elements:

- **A focused question and usable data.** Explain what you want to learn, where the data came from, what each observation represents, and any access, quality, or ethical limits that matter. For C, make the path from sources to the new dataset and its quality visible.
- **An ML approach your team understands.** Build and run the route-specific method or pipeline described above. Explain the important choices you made and distinguish your work from code, models, or ideas you borrowed.
- **Evidence that can challenge your idea.** Use evaluation that fits the question and the way the data were collected. Compare predictive results with a simple baseline; an exploratory or unsupervised extension may instead need a control, stability check, or other convincing reference. Explain your metric or diagnostic, guard against leakage, and avoid drawing a broad conclusion from one favorable run or example.
- **A conclusion with boundaries.** Show the main results in readable figures, tables, or examples. Investigate at least one informative error, failure, uncertainty, tradeoff, or setting where the conclusion may not hold. Say what you learned, what remains uncertain, and what you would try next.

- **Results someone else can reproduce.** The repository must connect the reported results to code, data instructions, and documented run steps. For C, this includes rebuilding or verifying the cleaned dataset from permitted inputs.

Choose experiments that help answer *your* question. There is no required number of models, datasets, or experiments. If your plan changes, briefly explain why.

How the project develops

These checkpoints give you feedback while there is still time to change course. **Every checkpoint and submission row below requires a new, dated entry in the root `README.md` of your team's GitHub repository.** The entry must describe the requested information in the right-hand column and link to the relevant code, results, report, or presentation materials. The checkpoint outline gives you an idea of where you should be in the project, but some flexibility is allowed so long as you make an update to your `README.md` file. Think of these more as journal entries; what progress has been made up to this point? Be sure to date your entries. Keep earlier entries so the README shows how the project developed. Supporting details may live elsewhere in the repository, but a project note or GitHub issue does not replace the README entry. Commit the entry and submit the repository URL with that commit in Canvas at each graded deadline. Unless a class activity is specified, deadlines are 11:59 PM Mountain Time.

Date	Checkpoint	Required README entry
Oct 20	Team and directions	Name the team and describe two possible questions, routes, and data sources. Start the repository. If you need help finding a team, contact the instructor.
Oct 27	Project plan	State the chosen route and question, data, initial reference, proposed new component (A), competition and pipeline decision (B), or dataset sources and construction plan (C). Explain the evaluation plan, any risks, and a smaller backup scope. Use the project-plan-template.md if helpful.
Nov 5	Data and reference result	Explain how data are obtained or prepared, the evaluation plan, and the first runnable result or baseline; link the relevant code and output. B teams should describe local validation; C teams should describe their first collected and cleaned sample.
Nov 12	Working investigation	Link the working code and summarize an initial comparison or diagnostic: the new component (A), raw-data-to-submission pipeline

Date	Checkpoint	Required README entry
		and first valid submission (B), or source-to-clean-data pipeline, quality checks, and initial ML result (C).
Nov 19	Evidence and interpretation	Summarize a meaningful comparison for A or B, or evidence about both data quality and ML usefulness for C. For B, explain the design change between pipelines and any improvement or degradation seen so far. Describe an informative failure or limitation and what the evidence changed in your thinking. Preliminary results are welcome.
Dec 3	Reproduction draft	Link the code-and-results-complete draft, identify the main result and known gaps, and give exact setup and run commands using the committed `uv` environment files. B teams should link both valid Kaggle submissions and the commands that generated them.
Dec 8 or 10	Presentation	Link the presentation materials and state the main findings before your assigned class meeting. Present during that meeting.
Dec 10	Final submission	Identify the final commit or tag and link the completed `report.md`, main results, and presentation materials. Submit the repository URL and commit or tag in Canvas by 11:59 PM Mountain Time.

The checkpoints assess reasonable progress and how you respond to feedback. They do not require your first idea to succeed or your final conclusion to match your proposal. There is no graded deliverable over Thanksgiving break, November 25-29.

The final GitHub repository

The repository is the main project artifact. Organize it in a way that fits your work. It should contain:

- a **README** that introduces the question and main finding, maps the repository, and gives exact setup, data-access, and run instructions;
- the code and configuration used for the main analysis, plus a **reproducible `uv` environment**: commit `pyproject.toml` and `uv.lock`, and state the required Python version in the README or a committed `.python-version` file;
- saved figures, tables, outputs, or examples to support the report and presentation; and
- a completed `report.md` based on the Markdown report template, linked from the repository README, and the presentation materials.

For route C, also include a short description of the dataset's schema, sources, collection date or version, permitted access, cleaning decisions, and quality checks. The repository must contain the scraping or collection scripts used for the reported dataset, even if the raw data cannot be shared. Document the commands that run those scripts and build the cleaned data from accessible inputs; include a shareable data snapshot or small fixture when source terms permit it. The ML run should use the cleaned output produced by this pipeline.

For route B, include the code, configurations, and run commands for **both** pipelines. Link each valid Kaggle submission to the pipeline and commit that produced it, and record its local validation and Kaggle result. A reader should be able to regenerate each submission file from the documented data access steps.

The written project report

Copy the report template into the root of your team's repository as `report.md`, fill it in with your own writing, commit it, and link it from the root README. Submit the completed report in your repository. This is an account of your project experience, not a publication-style technical paper. This is an opportunity to explain your understanding of the project and your outcomes. It is expected that you do **not** use AI for this part, other than for grammatical polishing. Tell readers why you chose the question, the main steps you took, the choices and challenges that shaped the work, and the final outcomes. Explain what worked, what did not, what you learned, and what remains uncertain. Include enough detail and links to code, data, figures, or results for a reader to follow your account, along with each teammate's contributions and your sources. The README carries the exact setup and run instructions, so the report can link to them.

There is **no page or word limit**. Use the length needed to fully describe your project, without padding it with raw logs or a list of every minor experiment. You may adapt the template headings to fit your story as long as the report covers the substance above.

Verify the environment

Before the December 3 review and again before final submission, **verify the environment from a fresh clone**. Run `uv sync`, then use `uv run` to execute your documented test, small demonstration, or representative-result command. Confirm that it finishes and produces the expected output; record the command and outcome in the corresponding dated README entry. The committed environment files recreate the local `.venv/` directory; do not commit that directory itself. A reader should be able to obtain permitted data and rerun a representative result without hidden notebook state or personal file paths. If the full run is costly, provide a small runnable path and record the configuration and

provenance of the full result. Do not commit credentials, restricted data, or unnecessary large files. If data cannot be shared, explain how an authorized reader can access them and provide a small permissible example when possible.

Presentation and collaboration

Plan for about **12 - 15 minutes** for the presentation. Slides, a live demonstration, or both are welcome. Focus on the question, the choices that mattered, the evidence, and what you learned. Every member should present a meaningful part and be able to discuss the shared project. Have your presentation materials ready for your assigned class meeting and add them to the final repository.

All teammates should make substantive technical or analytical contributions. You may divide tasks, but review each other's work and keep the main decisions understandable to the whole team. Cite datasets, papers, public notebooks, packages, and pretrained models. Follow data licenses and competition rules. If your idea involves collecting new information from people or using private or identifying data, discuss it with the instructor before proceeding.

AI use

AI tools may help you conceive project ideas, write or debug code, and polish work your team has already created. **Your team must create the final report and presentation slides, including their outlines, interpretation, and conclusions. Do not use AI to generate those deliverables.** I want to hear your own take on what happened in the project. Verify AI-assisted code and claims, and briefly disclose material AI help in `report.md`.

Grading

The project is worth **100 points**:

Component	Points
Checkpoints	30
Reproducible workflow	20
Presentation	20
Written project report	30

Component	Points
Total	**100**

The rubric explains how these points are judged; the presentation scoring sheet gives the detailed 20-point presentation criteria. The workflow score covers the working code, data-to-result path, and fresh-clone setup. The presentation and report scores reward your team's own account of its choices, evidence, challenges, and outcomes. Quality of reasoning and clear communication matter more than model count, compute spent, or leaderboard position.